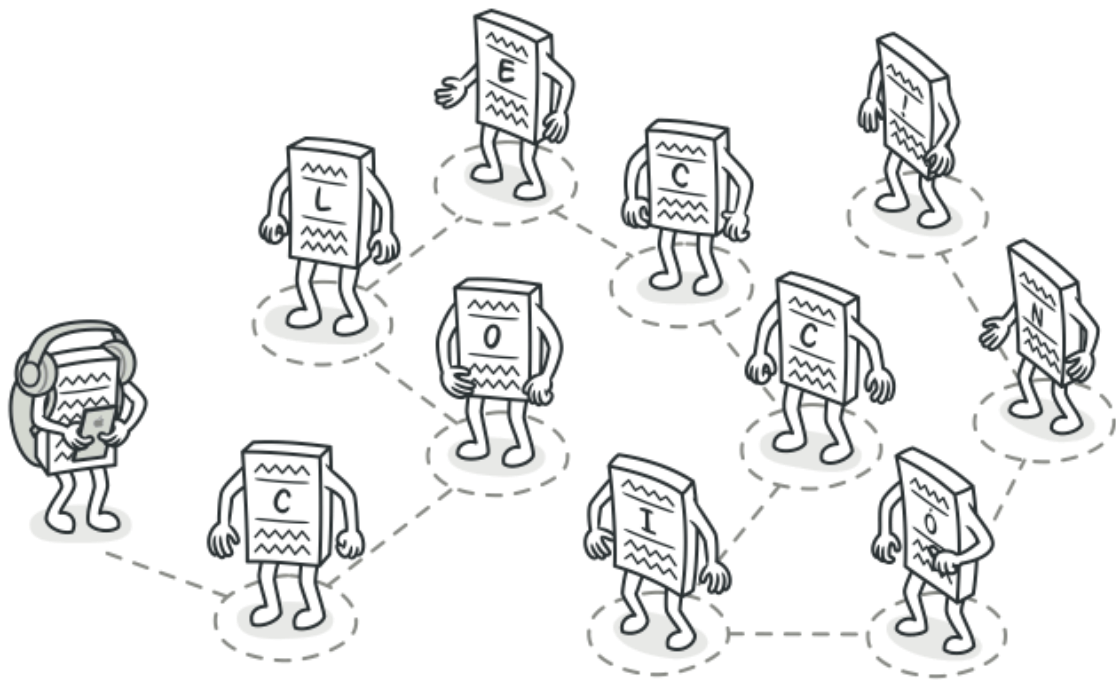


# Iterator

También llamado: Iterador

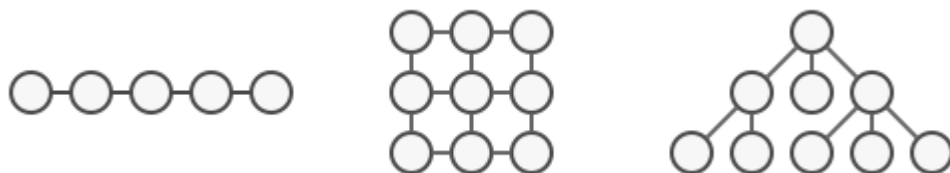
## Propósito

**Iterator** es un patrón de diseño de comportamiento que te permite recorrer elementos de una colección sin exponer su representación subyacente (lista, pila, árbol, etc.).



## Problema

Las colecciones son de los tipos de datos más utilizados en programación. Sin embargo, una colección tan solo es un contenedor para un grupo de objetos.



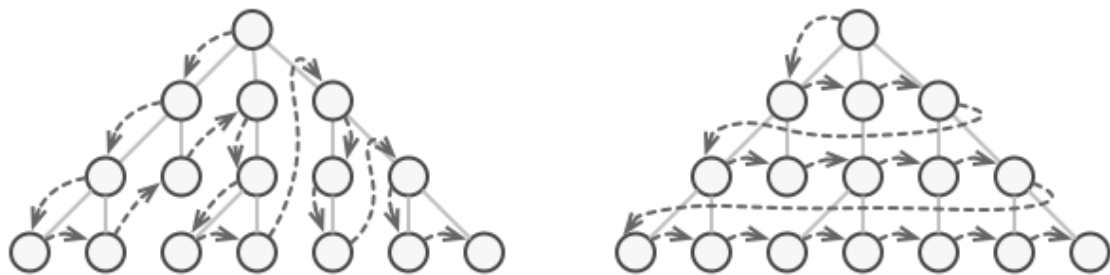
Varios tipos de colecciones.

La mayoría de las colecciones almacena sus elementos en simples listas, pero algunas de ellas se basan en pilas, árboles, grafos y otras estructuras complejas de datos.

Independientemente de cómo se estructure una colección, debe aportar una forma de acceder a sus elementos de modo que otro código pueda utilizar

dichos elementos. Debe haber una forma de recorrer cada elemento de la colección sin acceder a los mismos elementos una y otra vez.

Esto puede parecer un trabajo sencillo si tienes una colección basada en una lista. En este caso sólo tienes que recorrer en bucle todos sus elementos. Pero, ¿cómo recorres secuencialmente elementos de una estructura compleja de datos, como un árbol? Por ejemplo, un día puede bastarte con un recorrido de profundidad de un árbol, pero, al día siguiente, quizá necesites un recorrido en anchura. Y, la semana siguiente, puedes necesitar otra cosa, como un acceso aleatorio a los elementos del árbol.

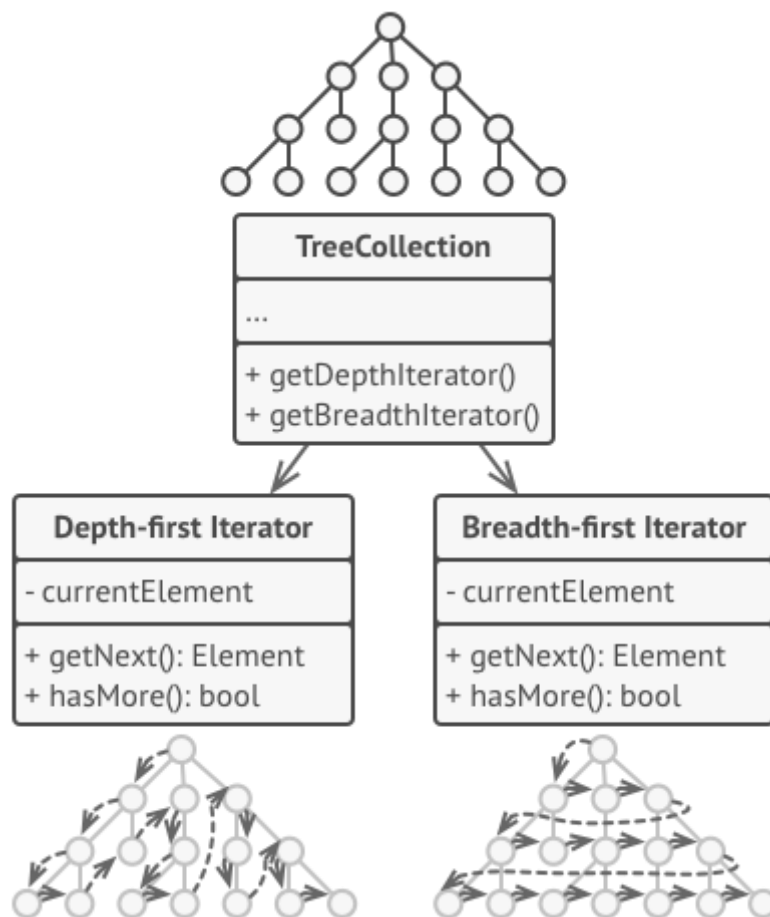


La misma colección puede recorrerse de varias formas diferentes. Añadir más y más algoritmos de recorrido a la colección nubla gradualmente su responsabilidad principal, que es el almacenamiento eficiente de la información. Además, puede que algunos algoritmos estén personalizados para una aplicación específica, por lo que incluirlos en una clase genérica de colección puede resultar extraño.

Por otro lado, el código cliente que debe funcionar con varias colecciones puede no saber cómo éstas almacenan sus elementos. No obstante, ya que todas las colecciones proporcionan formas diferentes de acceder a sus elementos, no tienes otra opción más que acoplar tu código a las clases de la colección específica.

## Solución

La idea central del patrón Iterator es extraer el comportamiento de recorrido de una colección y colocarlo en un objeto independiente llamado *iterador*.

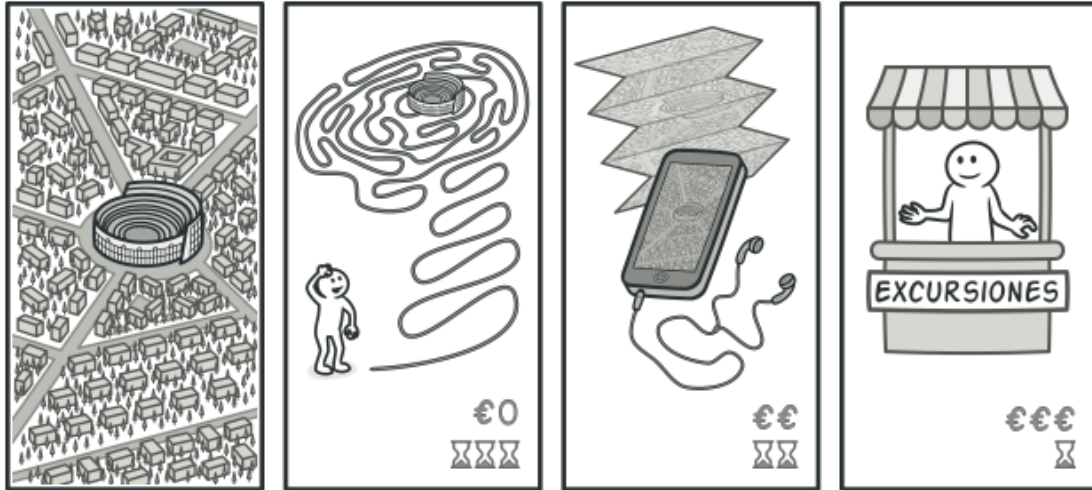


Los iteradores implementan varios algoritmos de recorrido. Varios objetos iteradores pueden recorrer la misma colección al mismo tiempo. Además de implementar el propio algoritmo, un objeto iterador encapsula todos los detalles del recorrido, como la posición actual y cuántos elementos quedan hasta el final. Debido a esto, varios iteradores pueden recorrer la misma colección al mismo tiempo, independientemente los unos de los otros.

Normalmente, los iteradores aportan un método principal para extraer elementos de la colección. El cliente puede continuar ejecutando este método hasta que no devuelva nada, lo que significa que el iterador ha recorrido todos los elementos.

Todos los iteradores deben implementar la misma interfaz. Esto hace que el código cliente sea compatible con cualquier tipo de colección o cualquier algoritmo de recorrido, siempre y cuando exista un iterador adecuado. Si necesitas una forma particular de recorrer una colección, creas una nueva clase iteradora sin tener que cambiar la colección o el cliente.

## Analogía en el mundo real



Varias formas de pasear por Roma.

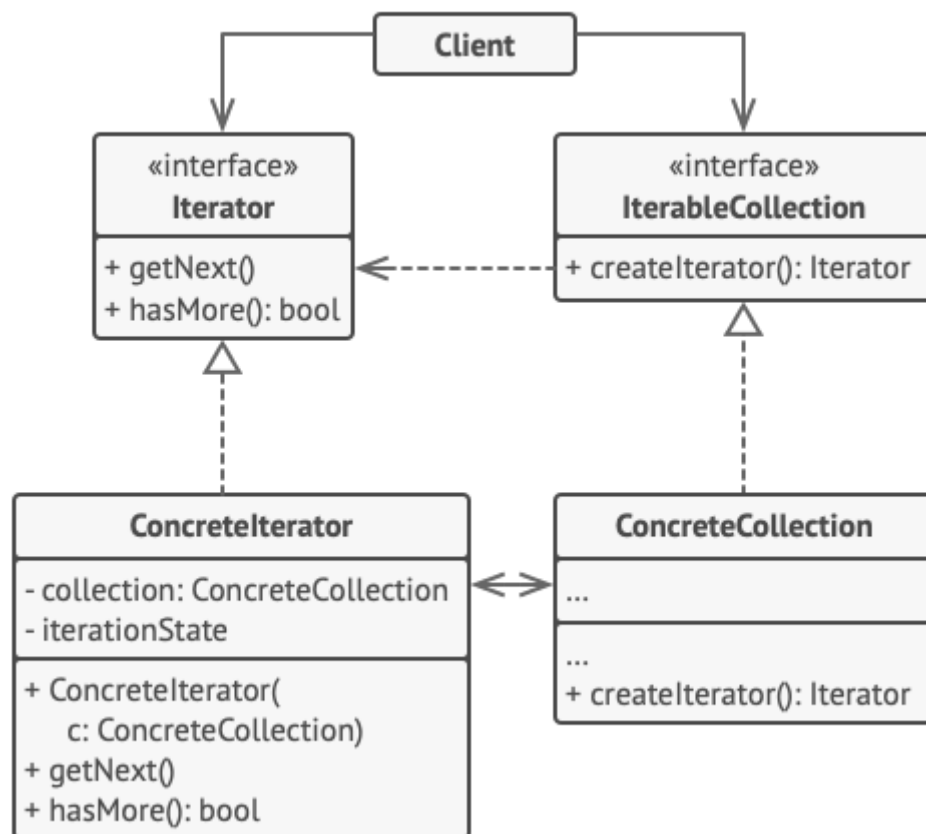
Planeas visitar Roma por unos días y ver todas sus atracciones y puntos de interés. Pero, una vez allí, podrías perder mucho tiempo dando vueltas, incapaz de encontrar siquiera el Coliseo.

En lugar de eso, podrías comprar una aplicación de guía virtual para tu smartphone y utilizarla para moverte. Es buena y barata y puedes quedarte en sitios interesantes todo el tiempo que quieras.

Una tercera alternativa sería dedicar parte del presupuesto del viaje a contratar un guía local que conozca la ciudad como la palma de su mano. El guía podría adaptar la visita a tus gustos, mostrarte las atracciones y contarte un montón de emocionantes historias. Eso sería más divertido pero, lamentablemente, también más caro.

Todas estas opciones —las direcciones aleatorias en tu cabeza, el navegador del smartphone o el guía humano—, actúan como iteradores sobre la amplia colección de visitas y atracciones de Roma.

## Estructura

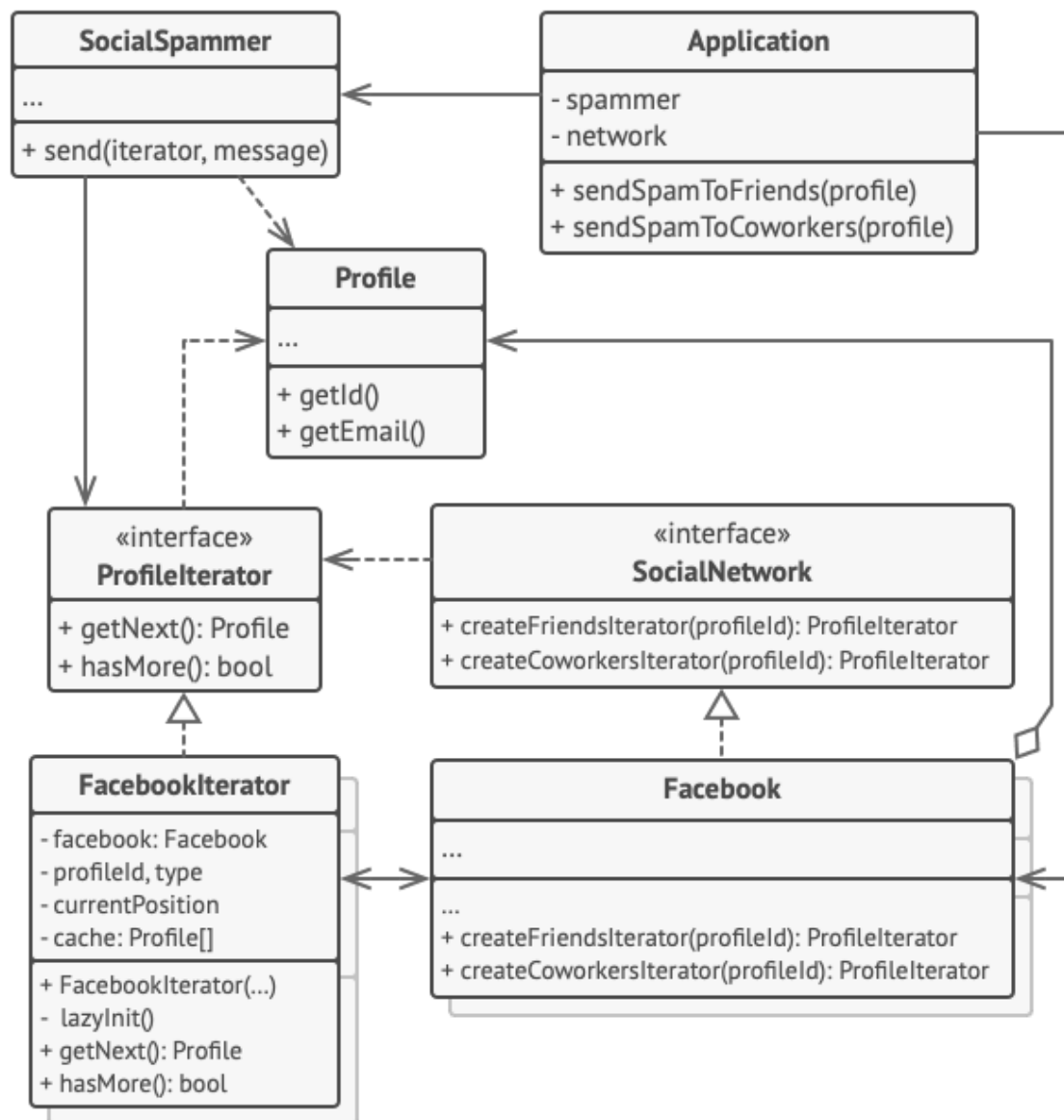


1. La interfaz **Iteradora** declara las operaciones necesarias para recorrer una colección: extraer el siguiente elemento, recuperar la posición actual, reiniciar la iteración, etc.
2. Los **Iteradores Concretos** implementan algoritmos específicos para recorrer una colección. El objeto iterador debe controlar el progreso del recorrido por su cuenta. Esto permite a varios iteradores recorrer la misma colección con independencia entre sí.
3. La interfaz **Colección** declara uno o varios métodos para obtener iteradores compatibles con la colección. Observa que el tipo de retorno de los métodos debe declararse como la interfaz iteradora de forma que las colecciones concretas puedan devolver varios tipos de iteradores.
4. Las **Colecciones Concretas** devuelven nuevas instancias de una clase iteradora concreta particular cada vez que el cliente solicita una. Puede que te estés preguntando: ¿dónde está el resto del código de la colección? No te preocupes, debe estar en la misma clase. Lo que pasa es que estos detalles no son fundamentales para el patrón en sí, por eso los omitimos.
5. El **Cliente** debe funcionar con colecciones e iteradores a través de sus interfaces. De este modo, el cliente no se acopla a clases concretas, permitiéndote utilizar varias colecciones e iteradores con el mismo código cliente.

Normalmente, los clientes no crean iteradores por su cuenta, en lugar de eso, los obtienen de las colecciones. Sin embargo, en algunos casos, el cliente puede crear uno directamente, como cuando define su propio iterador especial.

## Pseudocódigo

En este ejemplo, el patrón **Iterator** se utiliza para recorrer un tipo especial de colección que encapsula el acceso al grafo social de Facebook. La colección proporciona varios iteradores que recorren perfiles de distintas formas.



Ejemplo de iteración de perfiles sociales.

El iterador 'amigos' puede utilizarse para recorrer los amigos de un perfil dado. El iterador 'colegas' hace lo mismo, excepto que omite amigos que no trabajen en la misma empresa que la persona objetivo. Ambos iteradores implementan una interfaz común que permite a los clientes extraer perfiles sin profundizar en los detalles de la implementación, como la autenticación y el envío de solicitudes REST.

El código cliente no está acoplado a clases concretas porque sólo trabaja con colecciones e iteradores a través de interfaces. Si decides conectar tu aplicación a una nueva red social, sólo necesitas proporcionar nuevas clases de colección e iteradoras, sin cambiar el código existente.

// La interfaz de colección debe declarar un método fábrica para

```

// producir iteradores. Puedes declarar varios métodos si hay
// distintos tipos de iteración disponibles en tu programa.
interface SocialNetwork is
    method createFriendsIterator(profileId):ProfileIterator
    method createCoworkersIterator(profileId):ProfileIterator

// Cada colección concreta está acoplada a un grupo de clases
// iteradoras concretas que devuelve, pero el cliente no lo
// está, ya que la firma de estos métodos devuelve interfaces
// iteradoras.
class Facebook implements SocialNetwork is
    // ... El grueso del código de la colección debe ir aquí ...
    // Código de creación del iterador.
    method createFriendsIterator(profileId) is
        return new FacebookIterator(this, profileId, "friends")
    method createCoworkersIterator(profileId) is
        return new FacebookIterator(this, profileId, "coworkers")

// La interfaz común a todos los iteradores.
interface ProfileIterator is
    method getNext():Profile
    method hasMore():bool

// La clase iteradora concreta.
class FacebookIterator implements ProfileIterator is
    // El iterador necesita una referencia a la colección que
    // recorre.
    private field facebook: Facebook
    private field profileId, type: string

    // Un objeto iterador recorre la colección
    // independientemente de otro iterador, por eso debe
    // almacenar el estado de iteración.
    private field currentPosition
    private field cache: array of Profile

    constructor FacebookIterator(facebook, profileId, type) is
        this.facebook = facebook
        this.profileId = profileId
        this.type = type

    private method lazyInit() is
        if (cache == null)
            cache = facebook.socialGraphRequest(profileId, type)

    // Cada clase iteradora concreta tiene su propia
    // implementación de la interfaz iteradora común.
    method getNext() is
        if (hasMore())
            result = cache[currentPosition]
            currentPosition++
            return result

    method hasMore() is
        lazyInit()
        return currentPosition < cache.length

```

```
// Aquí tienes otro truco útil: puedes pasar un iterador a una
// clase cliente en lugar de darle acceso a una colección
// completa. De esta forma, no expones la colección al cliente.
//
// Y hay otra ventaja: puedes cambiar la forma en la que el
// cliente trabaja con la colección durante el tiempo de
// ejecución pasándole un iterador diferente. Esto es posible
// porque el código cliente no está acoplado a clases iteradoras
// concretas.
class SocialSpammer is
    method send(iterator: ProfileIterator, message: string) is
        while (iterator.hasMore())
            profile = iterator.getNext()
            System.sendEmail(profile.getEmail(), message)

// La clase Aplicación configura colecciones e iteradores y
// después los pasa al código cliente.
class Application is
    field network: SocialNetwork
    field spammer: SocialSpammer

    method config() is
        if working with Facebook
            this.network = new Facebook()
        if working with LinkedIn
            this.network = new LinkedIn()
        this.spammer = new SocialSpammer()

    method sendSpamToFriends(profile) is
        iterator = network.createFriendsIterator(profile.getId())
        spammer.send(iterator, "Very important message")

    method sendSpamToCoworkers(profile) is
        iterator = network.createCoworkersIterator(profile.getId())
        spammer.send(iterator, "Very important message")
```

## Aplicabilidad

**Utiliza el patrón Iterator cuando tu colección tenga una estructura de datos compleja a nivel interno, pero quieras ocultar su complejidad a los clientes (ya sea por conveniencia o por razones de seguridad).**

El iterador encapsula los detalles del trabajo con una estructura de datos compleja, proporcionando al cliente varios métodos simples para acceder a los elementos de la colección. Esta solución, además de ser muy conveniente para el cliente, también protege la colección frente a acciones descuidadas o maliciosas que el cliente podría realizar si trabajara con la colección directamente.

**Utiliza el patrón para reducir la duplicación en el código de recorrido a lo largo de tu aplicación.**

El código de los algoritmos de iteración no triviales tiende a ser muy voluminoso. Cuando se coloca dentro de la lógica de negocio de una



aplicación, puede nublar la responsabilidad del código original y hacerlo más difícil de mantener. Mover el código de recorrido a iteradores designados puede ayudarte a hacer el código de la aplicación más breve y limpio.

**Utiliza el patrón Iterator cuando quieras que tu código pueda recorrer distintas estructuras de datos, o cuando los tipos de estas estructuras no se conozcan de antemano.**

El patrón proporciona un par de interfaces genéricas para colecciones e iteradores. Teniendo en cuenta que ahora tu código utiliza estas interfaces, seguirá funcionando si le pasas varios tipos de colecciones e iteradores que implementen esas interfaces.

## Cómo implementarlo

1. Declara la interfaz iteradora. Como mínimo, debe tener un método para extraer el siguiente elemento de una colección. Por conveniencia, puedes añadir un par de métodos distintos, como para extraer el elemento previo, localizar la posición actual o comprobar el final de la iteración.
2. Declara la interfaz de colección y describe un método para buscar iteradores. El tipo de retorno debe ser igual al de la interfaz iteradora. Puedes declarar métodos similares si planeas tener varios grupos distintos de iteradores.
3. Implementa clases iteradoras concretas para las colecciones que quieras que sean recorridas por iteradores. Un objeto iterador debe estar vinculado a una única instancia de la colección. Normalmente, este vínculo se establece a través del constructor del iterador.
4. Implementa la interfaz de colección en tus clases de colección. La idea principal es proporcionar al cliente un atajo para crear iteradores personalizados para una clase de colección particular. El objeto de colección debe pasarse a sí mismo al constructor del iterador para establecer un vínculo entre ellos.
5. Repasa el código cliente para sustituir todo el código de recorrido de la colección por el uso de iteradores. El cliente busca un nuevo objeto iterador cada vez que necesita recorrer los elementos de la colección.

## Pros y contras

- *Principio de responsabilidad única.* Puedes limpiar el código cliente y las colecciones extrayendo algoritmos de recorrido voluminosos y colocándolos en clases independientes.
- *Principio de abierto/cerrado.* Puedes implementar nuevos tipos de colecciones e iteradores y pasarlos al código existente sin descomponer nada.
- Puedes recorrer la misma colección en paralelo porque cada objeto iterador contiene su propio estado de iteración.
- Por la misma razón, puedes retrasar una iteración y continuar cuando sea necesario.
- Aplicar el patrón puede resultar excesivo si tu aplicación funciona únicamente con colecciones sencillas.

- Utilizar un iterador puede ser menos eficiente que recorrer directamente los elementos de algunas colecciones especializadas.

## Relaciones con otros patrones

- Puedes utilizar **Iteradores** para recorrer árboles **Composite**.
- Puedes utilizar el patrón **Factory Method** junto con el **Iterator** para permitir que las subclases de la colección devuelvan distintos tipos de iteradores que sean compatibles con las colecciones.
- Puedes usar **Memento** junto con **Iterator** para capturar el estado de la iteración actual y reanudarla si fuera necesario.
- Puedes utilizar **Visitor** junto con **Iterator** para recorrer una estructura de datos compleja y ejecutar alguna operación sobre sus elementos, incluso aunque todos tengan clases distintas.